

techBuilder

Phase A — Frontend Implementation Guide

Complete Page-by-Page Implementation Blueprint

Every Screen. Every Component. Every State. Every Role.

Version 1.0 — March 2026

For: Development Team + Technical Stakeholders

Stack: [Next.js 16](#) + [Tailwind CSS v4](#) + [shadcn/ui](#) + [Mock Data](#)

Table of Contents

Table of Contents.....	2
Part 1: Project Foundation & Setup	6
1.1 Exact Tech Stack for Phase A Frontend	6
1.2 Frontend Folder Structure.....	6
1.3 Role-Based Routing Architecture.....	8
How Role Routing Works	8
Middleware Logic (Phase A — Mock Auth)	9
Dev-Only Role Selector Page	9
1.4 Design System — Visual Language.....	10
Color Palette	10
Typography Scale	11
Spacing System	11
Component Patterns — The 5 Core Patterns.....	12
1.5 Mock Data Architecture	13
Mock Data Strategy: The Service Layer Pattern.....	13
Mock Data Entity Schema.....	13
Entity: Company (Tenant)	13
Entity: User / Worker	13
Entity: Project / Site.....	14
Entity: Vehicle	14
Entity: Attendance Record	15
Entity: Vehicle Log	15
Entity: Fuel Entry.....	16
Entity: Expense	16
Entity: Photo / Media	17
Entity: Task	17
Entity: Daily Report	18
Entity: Material Usage	18
Entity: Issue	19
Entity: Notification	19
Mock Data Volume.....	20
Part 2: Worker Role — 6 Pages (Simplest UI).....	21
Worker Design Principles.....	21
2.1 Worker Home Screen	21
Screen Layout (Top to Bottom)	21

Action Card Details	22
State Machine: Clock In/Out Toggle.....	23
Component Tree	23
Mock Data Consumed.....	23
UI States.....	23
Accessibility Notes	24
Responsive Behavior	24
Hindi / English Implementation.....	24
Implementation Priority & Estimated Time.....	25
2.2 Worker Clock-In Page.....	26
Screen Layout.....	26
Interaction Flow.....	26
Error States.....	26
Component Tree	27
Mock Data Written	27
2.3 Worker Photo Upload Page	28
Screen Layout.....	28
Photo Watermark Specification	28
Phase A Implementation (Web Mock)	29
2.4 Worker View Task Page	30
Screen Layout.....	30
2.5 Worker Attendance History Page.....	31
Screen Layout.....	31
2.6 Worker Profile Page.....	32
2.7 Worker Leave Request Page.....	32
Form Fields.....	32
Part 3: Driver Role — 8 Pages	33
Driver Design Principles.....	33
3.1 Driver Home Screen	33
Screen Layout.....	33
Vehicle Type Branching	33
3.2 Driver Start Day Log	35
Form Fields.....	35
Validation Rules	35
3.3 Driver End Day Log	35
3.4 Driver Fuel Entry.....	36

Form Fields.....	36
3.5 Driver Expense Entry.....	37
3.6 Driver Vehicle Issue Report	37
3.7 Driver Daily Summary.....	37
3.8 Driver Photo Upload	37
Part 4: Worker-Head (Mistri) Role — 8 Pages.....	38
Worker-Head Design Principles	38
4.1 Worker-Head Home Screen.....	38
Screen Layout.....	38
4.2 Worker-Head Crew Attendance Page	39
Screen Layout.....	39
4.3 Worker-Head Upload on Behalf	40
Interaction Flow.....	40
4.4-4.8 Remaining Worker-Head Pages	41
Part 5: Sub-Manager Role — 11 Pages	42
Sub-Manager Design Principles.....	42
5.1 Sub-Manager Home / Site Dashboard	42
Screen Layout.....	42
5.2 Sub-Manager Attendance Page.....	44
Approval Mode.....	44
Direct Mode	44
5.3-5.11 Remaining Sub-Manager Pages.....	45
Part 6: Manager (Company Owner) Role — 14 Pages.....	46
Manager Design Principles	46
6.1 Manager Dashboard (Home)	46
Desktop Layout (3-column)	46
Mobile Layout (Single Column)	46
KPI Cards	46
Charts	47
6.2-6.14 Remaining Manager Pages	48
Part 7: Super Admin & Shared Pages	50
7.1 Super Admin — 5 Pages (Desktop Only).....	50
7.2 Shared Pages — Used by All Roles	50
Part 8: Implementation Schedule — Week-by-Week.....	51
Total Estimates Summary	51
Part 9: Shared Component Library.....	52

Part 10: Testing & Quality Assurance.....	54
10.1 Device Testing Matrix	54
10.2 Quality Checklist Per Page	54
Part 11: Conclusion — What You Have Now	56

Part 1: Project Foundation & Setup

This section covers everything you need to set up before writing a single page component. It defines the folder structure, tooling configuration, design system tokens, mock data architecture, and routing patterns that every page depends on.

1.1 Exact Tech Stack for Phase A Frontend

Every technology below has been chosen for a specific reason. This is not a buffet — these are the exact versions and libraries to install.

Technology	Version	Purpose	Why This One
Next.js	16.2+	Web framework (App Router)	Server Components, Turbopack dev server, middleware for role routing
React	19.2+	UI library	Ships with Next.js 16. React Compiler auto-memoizes.
Tailwind CSS	4.0+	Utility-first styling	Oxide engine (5x faster), CSS-first config, native shadcn/ui support
shadcn/ui	CLI v4	Component library (copy-paste)	50+ accessible components. You OWN the code. Built on Radix primitives.
TanStack Query	5.x	Server state / data fetching	Caching, background refetch, optimistic updates. Wraps our mock data now, real API later.
Zustand	5.x	Client state	3KB, no Provider needed. Stores role context, sidebar state, language preference.
React Hook Form	7.71+	Form handling	Uncontrolled forms = minimal re-renders. shadcn/ui Form built on it.
Zod	3.x	Schema validation	Shared between frontend validation + future backend validation.
next-intl	4.x	Hindi + English i18n	Server Component support, zero client bundle for translations.
Recharts	3.x	Dashboard charts	Declarative JSX API. Used in admin dashboard.
TanStack Table	8.x	Data tables	Headless, virtual scrolling, sorting, filtering. For admin views.
Lucide React	latest	Icons	Tree-shakable, consistent 24x24 icons. Used across all roles.
date-fns	3.x	Date formatting	Lightweight, tree-shakable. Hindi locale support.
TypeScript	5.7+	Type safety	Strict mode. Shared types in packages/shared/.

1.2 Frontend Folder Structure

The frontend lives in `apps/web/` inside the Turborepo monorepo. This structure supports role-based routing, shared components, and mock data that can be swapped for real API calls in Phase B without changing any page component.

Path	Purpose	Key Files
<code>apps/web/app/</code>	Next.js App Router pages	<code>layout.tsx</code> , <code>middleware.ts</code> , <code>(auth)/</code> , <code>(worker)/</code> , <code>(driver)/</code> , <code>(worker-head)/</code> , <code>(sub-manager)/</code> , <code>(manager)/</code> , <code>(admin)/</code>
<code>apps/web/app/(auth)/</code>	Authentication pages	<code>login/page.tsx</code> , <code>signup/page.tsx</code> , <code>onboarding/page.tsx</code> , <code>consent/page.tsx</code>
<code>apps/web/app/(worker)/</code>	Worker role pages	<code>home/page.tsx</code> , <code>photo/page.tsx</code> , <code>task/page.tsx</code> , <code>attendance/page.tsx</code> , <code>profile/page.tsx</code> , <code>leave/page.tsx</code>
<code>apps/web/app/(driver)/</code>	Driver role pages	<code>home/page.tsx</code> , <code>start-day/page.tsx</code> , <code>end-day/page.tsx</code> , <code>fuel/page.tsx</code> , <code>expense/page.tsx</code> , <code>issue/page.tsx</code> , <code>summary/page.tsx</code> , <code>photos/page.tsx</code>
<code>apps/web/app/(worker-head)/</code>	Worker-Head role pages	<code>home/page.tsx</code> , <code>crew-attendance/page.tsx</code> , <code>upload-behalf/page.tsx</code> , <code>progress/page.tsx</code> , <code>material/page.tsx</code> , <code>issues/page.tsx</code> , <code>daily-note/page.tsx</code> , <code>expense/page.tsx</code>
<code>apps/web/app/(sub-manager)/</code>	Sub-Manager role pages	<code>home/page.tsx</code> , <code>attendance/page.tsx</code> , <code>photos/page.tsx</code> , <code>report/page.tsx</code> , <code>tasks/page.tsx</code> , <code>expenses/page.tsx</code> , <code>progress/page.tsx</code> , <code>workers/page.tsx</code> , <code>vehicles/page.tsx</code> , <code>issues/page.tsx</code> , <code>material-request/page.tsx</code>
<code>apps/web/app/(manager)/</code>	Manager role pages	<code>home/page.tsx</code> , <code>sites/page.tsx</code> , <code>sites/[id]/page.tsx</code> , <code>attendance/page.tsx</code> , <code>fleet/page.tsx</code> , <code>feed/page.tsx</code> , <code>expenses/page.tsx</code> , <code>approve/page.tsx</code> , <code>workers/page.tsx</code> , <code>vehicles/page.tsx</code> , <code>reports/page.tsx</code> , <code>assign/page.tsx</code> , <code>notifications/page.tsx</code>
<code>apps/web/app/(admin)/</code>	Super Admin pages	<code>home/page.tsx</code> , <code>tenants/page.tsx</code> , <code>billing/page.tsx</code> , <code>audit/page.tsx</code> , <code>backups/page.tsx</code>
<code>apps/web/components/</code>	Shared UI components	<code>ui/</code> (shadcn), <code>layout/</code> (shells, navs), <code>forms/</code> (reusable form fields), <code>cards/</code> (data display cards)
<code>apps/web/components/ui/</code>	shadcn/ui components	<code>button.tsx</code> , <code>input.tsx</code> , <code>card.tsx</code> , <code>dialog.tsx</code> , <code>table.tsx</code> , <code>select.tsx</code> , <code>badge.tsx</code> , <code>calendar.tsx</code> , etc.
<code>apps/web/components/layout/</code>	Layout shells per role	<code>worker-shell.tsx</code> , <code>driver-shell.tsx</code> , <code>manager-shell.tsx</code> , <code>sidebar.tsx</code> , <code>mobile-nav.tsx</code> , <code>header.tsx</code>

Path	Purpose	Key Files
apps/web/components/forms/	Reusable form components	expense-form.tsx, photo-capture.tsx, attendance-marker.tsx, task-form.tsx, vehicle-log-form.tsx
apps/web/components/cards/	Data display cards	kpi-card.tsx, worker-card.tsx, vehicle-card.tsx, expense-card.tsx, activity-card.tsx, site-card.tsx
apps/web/lib/	Utilities and services	mock-data.ts, api.ts, auth.ts, utils.ts, constants.ts, types.ts
apps/web/lib/mock/	Mock data files	workers.json, vehicles.json, projects.json, attendance.json, expenses.json, fuel-logs.json, photos.json, tasks.json
apps/web/lib/hooks/	Custom React hooks	useAuth.ts, useRole.ts, useMockData.ts, useMediaUpload.ts, useOfflineQueue.ts
apps/web/lib/stores/	Zustand stores	auth-store.ts, ui-store.ts, language-store.ts
apps/web/messages/	i18n translation files	en.json, hi.json
apps/web/public/	Static assets	logos, icons, placeholder images
packages/shared/	Shared across web + mobile + API	types/, schemas/ (Zod), constants/ (roles, categories, status enums)

1.3 Role-Based Routing Architecture

techBuilder serves 6 completely different UIs from the same Next.js application. The routing system must detect who the user is and show them only their role-specific pages. In Phase A (mock data), we simulate this with a role selector instead of real authentication.

How Role Routing Works

Next.js App Router uses route groups (folders with parentheses) to organize pages by role WITHOUT affecting the URL. The middleware reads the user's role from a cookie/store and redirects them to the correct route group.

Role	Route Group	Home URL	Total Pages	Navigation Type
Worker	(worker)/	/home	6	Bottom tab bar (3-4 tabs, very simple)
Driver	(driver)/	/home	8	Bottom tab bar (4 tabs: Home, Fuel, Expense, Photos)
Worker-Head	(worker-head)/	/home	8	Bottom tab bar (4 tabs: Home, Crew, Upload, Report)
Sub-Manager	(sub-manager)/	/home	11	Bottom tab bar + hamburger menu for less frequent pages

Role	Route Group	Home URL	Total Pages	Navigation Type
Manager	(manager)/	/home	14	Sidebar navigation (desktop), bottom tab + drawer (mobile)
Super Admin	(admin)/	/home	5	Sidebar navigation (desktop only — admin is desktop-first)

Middleware Logic (Phase A — Mock Auth)

In Phase A, instead of real authentication, we use a role-selector page at `/dev/select-role` that sets a cookie. The middleware reads this cookie and redirects accordingly. In Phase B, this cookie will be replaced by a JWT token containing the user's role.

```
// middleware.ts — Simplified for Phase A
// 1. Read role from cookie: 'techbuilder-role'
// 2. If no role → redirect to /dev/select-role
// 3. If role=worker and path starts with /(manager) → redirect to /home
// 4. Each role group has its own layout.tsx with role-specific navigation
```

Dev-Only Role Selector Page

A special page at `/dev/select-role` exists ONLY in development. It shows 6 buttons — one per role. Clicking a role sets the cookie and redirects to that role's home page. This lets any developer instantly switch between roles during development.

Button	Sets Role	Redirects To	Sample User
Worker	worker	/home (worker layout)	Ramesh Kumar — Mason, Site Greenfield
Driver	driver	/home (driver layout)	Suresh Yadav — Truck driver, Vehicle MH-04-AB-1234
Worker-Head	worker-head	/home (worker-head layout)	Mistri Raju — Lead mason, Crew of 12
Sub-Manager	sub-manager	/home (sub-manager layout)	Priya Singh — Site manager, Greenfield Residency
Manager	manager	/home (manager layout)	Vikram Patel — Company owner, 3 active sites
Super Admin	admin	/home (admin layout)	Platform admin — sees all tenants

1.4 Design System — Visual Language

Every pixel in techBuilder follows a consistent design system. This section defines colors, typography, spacing, and component patterns that apply across ALL roles. Role-specific customizations layer on top of this base.

Color Palette

The color system uses Tailwind CSS v4 custom properties defined in the global CSS. Colors are semantic — you use 'primary', 'destructive', etc., not raw hex values.

Token	Light Mode	Dark Mode	Usage
--color-primary	#1A5276 (deep blue)	#3498DB	Primary buttons, active tabs, links, selected states
--color-primary-foreground	#FFFFFF	#FFFFFF	Text on primary-colored backgrounds
--color-secondary	#F2F4F4 (light gray)	#2C3E50	Secondary buttons, subtle backgrounds
--color-accent	#2E86C1 (blue)	#5DADE2	Highlights, focus rings, accent elements
--color-destructive	#E74C3C (red)	#E74C3C	Delete buttons, error states, critical alerts
--color-success	#27AE60 (green)	#2ECC71	Clock-in confirmation, success toasts, approved badges
--color-warning	#F39C12 (amber)	#F1C40F	Pending states, warning badges, fuel alerts
--color-muted	#95A5A6 (gray)	#7F8C8D	Disabled states, placeholder text, secondary info
--color-background	#FFFFFF	#1A1A2E	Page background
--color-card	#FFFFFF	#16213E	Card surfaces, modal backgrounds
--color-border	#E5E7EB	#374151	Card borders, dividers, input borders
--role-worker	#27AE60 (green)	#2ECC71	Worker-specific accent (clock-in button, worker badges)
--role-driver	#F39C12 (amber)	#F1C40F	Driver-specific accent (vehicle cards, fuel indicators)
--role-manager	#2E86C1 (blue)	#5DADE2	Manager-specific accent (dashboard KPIs, approve buttons)

Typography Scale

Construction workers use low-end Android phones with small screens. Typography must be large enough to read in bright sunlight while wearing safety glasses. We use a larger base size than typical web apps.

Token	Size	Weight	Usage
text-xs	12px / 0.75rem	400	Timestamps, metadata, fine print
text-sm	14px / 0.875rem	400	Secondary labels, helper text, table cells
text-base	16px / 1rem	400	Body text, form labels, list items (DEFAULT)
text-lg	18px / 1.125rem	500	Card titles, section headers, important labels
text-xl	20px / 1.25rem	600	Page titles, KPI values, prominent numbers
text-2xl	24px / 1.5rem	700	Dashboard KPI numbers, hero stats
text-3xl	30px / 1.875rem	700	Large dashboard numbers (total expense, total workers)
text-4xl	36px / 2.25rem	800	Main KPI on manager dashboard (today's total)

Spacing System

Consistent spacing creates visual rhythm. We use Tailwind's default 4px grid: p-1 = 4px, p-2 = 8px, p-3 = 12px, p-4 = 16px, etc.

Context	Spacing	Tailwind Class	Example
Between form fields	16px	space-y-4	Clock-in form: GPS status, timestamp, submit button
Inside cards	16px padding	p-4	Worker card: photo, name, role, status
Between cards	12px	gap-3	Manager site list: cards stacked vertically
Between sections	24px	space-y-6	Dashboard sections: KPIs, chart, recent activity
Page padding (mobile)	16px horizontal	px-4	All mobile pages have consistent side padding
Page padding (desktop)	24-32px	px-6 lg:px-8	Desktop dashboard has wider breathing room
Bottom nav safe area	80px bottom padding	pb-20	Every mobile page reserves space for bottom nav bar
Touch target minimum	44x44px	min-h-11 min-w-11	All buttons, checkboxes, tappable elements

Component Patterns — The 5 Core Patterns

Every page in techBuilder is built from combinations of just 5 core UI patterns. Understanding these patterns means you can build any page in the app.

Pattern	Description	Example	Implementation Notes
Action Card	A large, tappable card with an icon, title, and optional subtitle. Used as the primary navigation element on role home screens.	Worker home: 'Clock IN' (green, large), 'Upload Photo' (blue), 'View Task' (gray). Driver home: 'Start Day' (green), 'Log Fuel' (amber), 'End Day' (red).	Touch target: minimum 80px height. Icon: 40x40px. Border radius: 16px. Active state: scale(0.97) on press. Color: role-specific accent.
Data Entry Form	A structured form with labeled fields, validation, and a submit button. Used for all data input: expenses, vehicle logs, material usage.	Fuel entry: Amount (number input), Liters (number input), Receipt photo (camera button), GPS (auto-captured, shown as badge). Submit button full-width at bottom.	All fields use React Hook Form + Zod validation. Error messages in red below field. Submit button disabled until form is valid. Offline: form submits to local queue, shows success toast immediately.
List View	A scrollable list of items with avatar/icon, title, subtitle, and status badge. Used for worker lists, expense lists, task lists, vehicle lists.	Sub-Manager worker list: Worker photo (40x40 circle), Name (bold), Role (muted), Attendance badge (green 'Present' / red 'Absent'). Tap to view detail.	Virtualized scrolling for 100+ items (TanStack Virtual). Pull-to-refresh gesture. Search bar at top (filters locally). Empty state with illustration.
KPI Dashboard	A grid of metric cards showing numbers, trends, and quick stats. Used on manager and sub-manager home screens.	Manager home: 4 KPI cards — Workers Today (152/180), Active Vehicles (12/15), Today's Expense (Rs.45,200), Active Sites (5). Each card has: number, label, trend arrow (up/down), tap to drill down.	Cards use CSS Grid: 2 columns on mobile, 4 on desktop. Numbers animate on load (count-up effect). Trend calculated from yesterday's value in mock data.
Activity Feed	A chronological list of events with type icons, descriptions, timestamps, and optional photo thumbnails. Used as the 'WhatsApp replacement' for managers.	Manager feed: '10:32 AM — Ramesh Kumar clocked in at Greenfield' (green dot), '10:45 AM — Suresh uploaded fuel receipt Rs.2,450' (amber dot, photo thumbnail), '11:15 AM — Priya flagged issue: Water pipe burst at Block B' (red dot, photo).	Infinite scroll with pagination (20 items per page). Filter tabs: All, Photos, Expenses, Issues. Real-time updates in Phase C via WebSocket. For now, mock data with timestamps.

1.5 Mock Data Architecture

In Phase A, ALL data comes from local JSON files. The mock data layer is designed so that replacing it with real API calls in Phase B requires changing ONLY the data-fetching functions — zero changes to page components or UI.

Mock Data Strategy: The Service Layer Pattern

Every page fetches data through a service layer (`lib/api.ts`), not directly from JSON files. In Phase A, these service functions read from JSON. In Phase B, they make HTTP requests to NestJS. Page components never know the difference.

```
// lib/api.ts - Phase A (mock data)
export async function getWorkerAttendance(workerId: string, month: string) {
  const data = await import('./mock/attendance.json');
  return data.filter(a => a.workerId === workerId && a.month === month);
}

// lib/api.ts - Phase B (real API) - SAME function signature
export async function getWorkerAttendance(workerId: string, month: string) {
  const res = await fetch(`/api/attendance?worker=${workerId}&month=${month}`);
  return res.json();
}
```

Mock Data Entity Schema

Below is every data entity needed for the frontend, with exact field definitions. These map directly to the future PostgreSQL schema.

Entity: Company (Tenant)

Field	Type	Description / Example
id	string (UUID)	Unique tenant identifier
name	string	'Vikram Construction Pvt Ltd'
logo	string (URL)	Company logo for branding
primaryColor	string (hex)	Brand color for white-labeling
ownerName	string	'Vikram Patel'
plan	enum	'free' 'basic' 'pro'
createdAt	ISO date	When the company signed up

Entity: User / Worker

Field	Type	Description / Example
id	string (UUID)	Unique user identifier
tenantId	string (UUID)	Which company they belong to
name	string	'Ramesh Kumar'
phone	string	'+91-9876543210'
role	enum	'worker' 'driver' 'worker-head' 'sub-manager' 'manager' 'admin'
photo	string (URL)	Profile photo URL
projectId	string (UUID)	Currently assigned project/site
crewId	string (UUID) null	Worker-head's crew (if applicable)
emergencyContact	string	Emergency phone number
dailyWage	number	500 (in Rs.) — for wage calculation
language	enum	'en' 'hi'
isActive	boolean	true if currently employed
joinedAt	ISO date	When they joined the company

Entity: Project / Site

Field	Type	Description / Example
id	string (UUID)	Unique project identifier
tenantId	string (UUID)	Which company owns this project
name	string	'Greenfield Residency Block B'
address	string	Full address for display
lat	number	GPS latitude for geofencing
lng	number	GPS longitude for geofencing
status	enum	'active' 'paused' 'completed'
budget	number	5000000 (Rs. 50 lakhs)
startDate	ISO date	Project start date
expectedEndDate	ISO date	Planned completion date
progress	number	0-100 (percentage complete)
managerId	string (UUID)	Sub-manager assigned to this site

Entity: Vehicle

Field	Type	Description / Example
id	string (UUID)	Unique vehicle identifier
tenantId	string (UUID)	Which company owns this vehicle
name	string	'Tata 407 Truck' or 'JCB 3DX'
type	enum	'km-type' 'hours-type'
plateNumber	string	'MH-04-AB-1234'
assignedDriverId	string (UUID) null	Current primary driver
assignedProjectId	string (UUID) null	Current site assignment
status	enum	'active' 'maintenance' 'idle'
rcPhoto	string (URL)	Registration certificate photo
insuranceExpiry	ISO date	When insurance expires
pucExpiry	ISO date	Pollution certificate expiry
lastServiceDate	ISO date	Last maintenance date

Entity: Attendance Record

Field	Type	Description / Example
id	string (UUID)	Unique record
tenantId	string (UUID)	Company
workerId	string (UUID)	Who this record is for
projectId	string (UUID)	Which site
date	ISO date	YYYY-MM-DD
clockIn	ISO datetime null	When they clocked in
clockOut	ISO datetime null	When they clocked out
clockInLat	number null	GPS at clock-in
clockInLng	number null	GPS at clock-in
clockOutLat	number null	GPS at clock-out
clockOutLng	number null	GPS at clock-out
status	enum	'present' 'absent' 'half-day' 'leave'
markedBy	string (UUID)	Who marked this (self, worker-head, or sub-manager)
approvedBy	string (UUID) null	Sub-manager who approved (if applicable)
approvalStatus	enum	'auto-approved' 'pending' 'approved' 'rejected'

Entity: Vehicle Log

Field	Type	Description / Example
id	string (UUID)	Unique log entry
tenantId	string (UUID)	Company
vehicleId	string (UUID)	Which vehicle
driverId	string (UUID)	Which driver logged this
projectId	string (UUID)	Which site the vehicle was working at
date	ISO date	YYYY-MM-DD
type	enum	'start' 'end'
reading	number	KM reading (for km-type) or Hour reading (for hours-type)
photo	string (URL)	Odometer/hour-meter photo
lat	number	GPS at time of log
lng	number	GPS at time of log
timestamp	ISO datetime	Exact time of capture

Entity: Fuel Entry

Field	Type	Description / Example
id	string (UUID)	Unique entry
tenantId	string (UUID)	Company
vehicleId	string (UUID)	Which vehicle
driverId	string (UUID)	Who filled fuel
date	ISO date	YYYY-MM-DD
amount	number	Rs. amount paid
liters	number	Liters filled
receiptPhoto	string (URL)	Receipt photo (mandatory)
lat	number	GPS at fuel station
lng	number	GPS at fuel station
timestamp	ISO datetime	Time of fill

Entity: Expense

Field	Type	Description / Example
id	string (UUID)	Unique expense
tenantId	string (UUID)	Company

Field	Type	Description / Example
projectId	string (UUID)	Which site
submittedBy	string (UUID)	Who logged this expense
category	enum	'material' 'fuel' 'labor' 'food' 'transport' 'repair' 'other'
amount	number	Rs. amount
description	string	What was this expense for
receiptPhoto	string (URL) null	Receipt photo (optional for some categories)
date	ISO date	YYYY-MM-DD
approvalStatus	enum	'pending' 'approved' 'rejected'
approvedBy	string (UUID) null	Who approved this
approvalNote	string null	Comment from approver

Entity: Photo / Media

Field	Type	Description / Example
id	string (UUID)	Unique photo
tenantId	string (UUID)	Company
projectId	string (UUID)	Which site
uploadedBy	string (UUID)	Who uploaded
onBehalfOf	string (UUID) null	If uploaded by worker-head on behalf of another worker
type	enum	'work-progress' 'issue' 'delivery' 'receipt' 'odometer' 'site-morning' 'site-evening'
url	string (URL)	Full-size image URL
thumbnailUrl	string (URL)	200px thumbnail URL
note	string null	Description or caption
lat	number	GPS latitude
lng	number	GPS longitude
timestamp	ISO datetime	When photo was taken

Entity: Task

Field	Type	Description / Example
id	string (UUID)	Unique task
tenantId	string (UUID)	Company

Field	Type	Description / Example
projectId	string (UUID)	Which site
assignedTo	string (UUID)	Worker or worker-head assigned
assignedBy	string (UUID)	Sub-manager who assigned
title	string	Task title ('Lay bricks for wall section C')
description	string null	Additional details
date	ISO date	Which day this task is for
status	enum	'pending' 'in-progress' 'completed'
completedAt	ISO datetime null	When task was marked done
priority	enum	'normal' 'high'

Entity: Daily Report

Field	Type	Description / Example
id	string (UUID)	Unique report
tenantId	string (UUID)	Company
projectId	string (UUID)	Which site
authorId	string (UUID)	Sub-manager or worker-head who wrote this
date	ISO date	YYYY-MM-DD
summary	string	Free-text summary of the day
workerCount	number	How many workers were present (auto-filled)
expenseTotal	number	Total expenses for the day (auto-filled)
progressUpdates	JSON	Array of {milestoneId, percentage} updates
issues	string[]	List of issue IDs reported today
morningPhotos	string[]	Photo IDs for morning site photos
eveningPhotos	string[]	Photo IDs for evening site photos

Entity: Material Usage

Field	Type	Description / Example
id	string (UUID)	Unique record
tenantId	string (UUID)	Company
projectId	string (UUID)	Which site
loggedBy	string (UUID)	Worker-head who logged this

Field	Type	Description / Example
materialType	enum	'cement' 'steel' 'sand' 'bricks' 'aggregate' 'paint' 'wood' 'pipes' 'electrical' 'other'
quantity	number	Amount used
unit	enum	'bags' 'kg' 'tons' 'pieces' 'meters' 'liters' 'sqft'
date	ISO date	YYYY-MM-DD
note	string null	Additional context

Entity: Issue

Field	Type	Description / Example
id	string (UUID)	Unique issue
tenantId	string (UUID)	Company
projectId	string (UUID)	Which site
reportedBy	string (UUID)	Who reported this
title	string	Short issue title
description	string	Detailed description
severity	enum	'low' 'medium' 'high' 'critical'
category	enum	'structural' 'material' 'safety' 'delay' 'equipment' 'other'
photos	string[]	Array of photo IDs
status	enum	'open' 'in-progress' 'escalated' 'resolved'
assignedTo	string (UUID) null	Who is handling this
resolution	string null	How it was resolved
reportedAt	ISO datetime	When issue was created
resolvedAt	ISO datetime null	When issue was closed

Entity: Notification

Field	Type	Description / Example
id	string (UUID)	Unique notification
tenantId	string (UUID)	Company
userId	string (UUID)	Who receives this notification
type	enum	'task-assigned' 'expense-approved' 'expense-rejected' 'attendance-approved' 'issue-reported'

Field	Type	Description / Example
		'issue-escalated' 'leave-approved' 'leave-rejected' 'vehicle-breakdown' 'general'
title	string	Notification headline
body	string	Notification detail text
referenceId	string (UUID) null	ID of related entity (task, expense, etc.)
isRead	boolean	Has user seen this
createdAt	ISO datetime	When notification was created

Mock Data Volume

The mock data files should contain enough records to make the UI feel realistic. Here are the recommended volumes:

Entity	Records	Rationale
Companies	1 (our demo company)	Phase A focuses on one tenant
Users/Workers	25-30	Mix: 15 workers, 5 drivers, 3 worker-heads, 2 sub-managers, 1 manager
Projects/Sites	3	Enough to test multi-site views without overwhelming
Vehicles	8	Mix: 4 trucks (km-type), 2 JCBs (hours-type), 1 crane, 1 mixer
Attendance (30 days)	750+	25 workers x 30 days. Mix of present, absent, half-day, leave.
Vehicle Logs (30 days)	480+	8 vehicles x 30 days x 2 (start + end)
Fuel Entries (30 days)	60-90	Average 2-3 fills per vehicle per month
Expenses (30 days)	100-150	Mix of all categories across 3 sites
Photos (30 days)	200-300	Site photos, progress, issues, receipts — use placeholder images
Tasks (30 days)	60-90	2-3 tasks per worker-head/worker per day
Daily Reports (30 days)	60	2 sub-managers x 30 days
Issues	15-20	Mix of open, in-progress, resolved across all severities
Notifications	50-80	Realistic notification feed for each role

Part 2: Worker Role — 6 Pages (Simplest UI)

The Worker interface is intentionally the simplest in the entire application. The target user is a construction laborer who may be using a smartphone for the first time. They may not read English. They may be wearing gloves. The phone may cost Rs.5,000-8,000 and have a cracked screen. Design for the worst case.

Worker Design Principles

- Maximum 3-4 large buttons on any screen. No small tap targets.
- Every button has an icon + text label. Never rely on icon alone.
- Primary actions use color-coded cards (green = clock in, red = clock out, blue = upload).
- Hindi is the DEFAULT language. English is secondary.
- No scrolling on the home screen. Everything above the fold.
- Success/error states use full-screen overlays with large checkmark or X icon.
- Load time must be under 2 seconds on a 3G connection.
- Works 100% offline — all actions queue locally and sync when network returns.

2.1 Worker Home Screen

URL: /home (within (worker) route group)

Purpose: The ONLY screen most workers will ever use. 3-4 large action cards for daily operations.

Who uses it: Construction laborers, masons, painters, electricians — anyone with the 'worker' role.

Frequency: Opened 2-4 times per day: morning clock-in, photo uploads during day, evening clock-out.

Screen Layout (Top to Bottom)

The screen is divided into 3 sections stacked vertically. No scrolling needed — everything fits above the fold on a 5-inch screen.

Section	Height	Content	Implementation
Header Bar	56px	Worker name, profile photo (32px circle), language toggle (HI/EN), notification bell with badge count	Sticky at top. Uses Zustand language store for toggle. Notification count from mock data.
Status Card	80px	Today's status: 'You are CLOCKED IN since 7:02 AM' (green) or 'You have NOT clocked in today' (amber). Shows current time.	Reads from today's attendance record in mock data. Uses date-fns for time formatting. Pulse animation on the status dot.

Section	Height	Content	Implementation
Action Cards Grid	Remaining	2x2 grid of large action cards. Each card: 48px icon + 16px text label + subtle description. Cards: (1) Clock IN/OUT, (2) Upload Photo, (3) Today's Task, (4) My Attendance	CSS Grid: grid-cols-2. Each card minimum 120px height. Tap → navigate to respective page. Clock IN card turns into Clock OUT card after clock-in (green → red).

Action Card Details

Card	Icon	Color	Label (Hindi)	Label (English)	Tap Action	State Variations
Clock IN	LogIn (Lucide)	Green (#27AE60)	□□□□□□ □□□□□	Clock In	Navigate to /clock-in	Disabled + gray after clock-in. Shows 'Already clocked in at 7:02 AM' text.
Clock OUT	LogOut (Lucide)	Red (#E74C3C)	□□□□□□ □□□□□	Clock Out	Navigate to /clock-out	Hidden before clock-in. Appears after clock-in replacing the Clock IN card.
Upload Photo	Camera (Lucide)	Blue (#2E86C1)	□□□□□□ □□□□□□	Upload Photo	Navigate to /photo	Always visible. Badge shows pending upload count if offline queue has items.
Today's Task	ClipboardList (Lucide)	Purple (#8E44AD)	□□ □□ □□□□	Today's Task	Navigate to /task	Shows task title preview if assigned. Shows 'No task' in muted text if none.
My Attendance	Calendar (Lucide)	Gray (#95A5A6)	□□□□□□ □□□□□□□□	My Attendance	Navigate to /attendance	Shows 'X days this month' as subtitle.

Card	Icon	Color	Label (Hindi)	Label (English)	Tap Action	State Variations
Profile	User (Lucide)	Gray (#95A5A6)	□□□□ □□□□□□□□	My Profile	Navigate to /profile	Shows in bottom nav, not in grid.

State Machine: Clock In/Out Toggle

The home screen's primary action card changes based on attendance state. This is the most important state logic on this page.

Current State	What User Sees	Card 1 Shows	Card 2 Shows
Not clocked in (morning)	Amber status: 'Not clocked in yet'	Clock IN (green, large, pulsing border)	Upload Photo (blue)
Clocked in (during day)	Green status: 'Clocked in since 7:02 AM'	Clock OUT (red)	Upload Photo (blue)
Clocked out (evening)	Gray status: 'Day complete. Worked 9h 15m'	Both cards disabled/grayed out	Upload Photo still active
On leave	Blue status: 'You are on leave today'	All cards disabled	Only Profile accessible

Component Tree

```

WorkerHomePage
├── WorkerHeader (name, photo, lang toggle, notification bell)
├── AttendanceStatusCard (today's clock-in status)
├── ActionCardGrid
│   ├── ActionCard (Clock IN or Clock OUT - conditional)
│   ├── ActionCard (Upload Photo)
│   ├── ActionCard (Today's Task)
│   └── ActionCard (My Attendance)
└── WorkerBottomNav (Home, Photo, Profile - 3 tabs)

```

Mock Data Consumed

- Current user profile: lib/mock/users.json (filtered by current mock user ID)
- Today's attendance: lib/mock/attendance.json (filtered by userId + today's date)
- Today's task: lib/mock/tasks.json (filtered by assignedTo + today's date)
- Notification count: lib/mock/notifications.json (filtered by userId + isRead=false)
- Pending upload count: from Zustand offline queue store (starts at 0 in mock)

UI States

State	What Shows	How to Trigger in Dev
Loading	Skeleton cards (3 gray pulsing rectangles in 2x2 grid). Skeleton status bar.	Add artificial 1s delay in mock data fetch.
Empty (no attendance)	Amber status card + Clock IN as primary action	Default state when no attendance record for today
Clocked in	Green status + Clock OUT card replaces Clock IN	Mock data has clockIn but no clockOut for today
Day complete	Gray status + disabled cards + summary stats	Mock data has both clockIn and clockOut
On leave	Blue status + all cards disabled	Mock attendance has status: 'leave'
Offline	Small orange banner at top: 'You are offline. Changes will sync later.'	Simulate with navigator.onLine = false or NetInfo mock
Error	Red toast at bottom: 'Something went wrong. Please try again.'	Throw error in mock data fetch

Accessibility Notes

- All action cards have aria-label with full description (e.g., 'Clock in to mark your attendance for today')
- Status card uses role='status' and aria-live='polite' for screen reader updates
- Color is never the ONLY indicator — icons and text labels accompany every color-coded element
- Minimum contrast ratio: 4.5:1 for all text on all backgrounds
- Focus order: Status → Clock IN → Upload Photo → Today's Task → My Attendance → Bottom Nav
- Touch targets: All cards are minimum 44x44px (we use 120px+ height cards, well above minimum)

Responsive Behavior

Breakpoint	Layout Change	Notes
< 360px (very small phone)	Action cards stack to 1 column. Each card full-width.	Rare but must work — some workers have very old phones.
360-640px (most phones)	2x2 grid of action cards. This is the PRIMARY design target.	95% of users will see this layout.
640-1024px (tablet)	2x2 grid with larger cards and more padding.	Site managers sometimes use tablets.
> 1024px (desktop)	Cards arranged in a row (4 cards). Wider status card.	Workers rarely use desktop, but should still work for demos.

Hindi / English Implementation

Every visible string on this page must be in the translation file. No hardcoded English strings. Here are the translation keys for this page:

Translation Key	English	Hindi
worker.home.title	Home	□□□
worker.home.status.notClockedIn	You have not clocked in today	□□ □□□□ □□□□□□ □□□□ □□□□
worker.home.status.clockedIn	Clocked in since {time}	{time} □□ □□□□□□ □□□ □□
worker.home.status.dayComplete	Day complete. Worked {hours}	□□□ □□□□□ {hours} □□□ □□□□
worker.home.status.onLeave	You are on leave today	□□ □□□□ □□□□□□ □□
worker.home.clockIn	Clock In	□□□□□□□□ □□□□□
worker.home.clockOut	Clock Out	□□□□□□□ □□□□
worker.home.uploadPhoto	Upload Photo	□□□□□□ □□□□□
worker.home.todaysTask	Today's Task	□□ □□ □□□
worker.home.myAttendance	My Attendance	□□□□ □□□□□□□
worker.home.profile	My Profile	□□□□ □□□□□□□□□
worker.home.offline	You are offline. Changes will sync later.	□□ □□□□□□□ □□□□ □□□□□ □□□ □□□ □□□□ □□□□□□
worker.home.alreadyClockedIn	Already clocked in at {time}	{time} □□ □□□□□□□ □□□□ □□ □□□ □□
worker.home.daysThisMonth	{count} days this month	□□ □□□□□ {count} □□□
worker.home.noTask	No task assigned	□□□ □□□ □□□□ □□□□ □□□

Implementation Priority & Estimated Time

Priority: P0 — This is the FIRST page to build for the entire project.

Estimated time: 4-6 hours for a senior developer (including responsive + Hindi).

Dependencies: shadcn/ui Card + Button components, Zustand auth store, mock attendance data, next-intl setup.

Build order: 1) Static layout with hardcoded data → 2) Connect mock data → 3) Add state machine → 4) Hindi translations → 5) Responsive → 6) Accessibility

2.2 Worker Clock-In Page

URL: /clock-in

Purpose: Capture the worker's clock-in event with GPS location and timestamp. This is a confirmation page, not a form — the worker taps one button and they're done.

Who uses it: All workers, every morning.

Frequency: Once per day, typically between 6-8 AM.

Screen Layout

Section	Height	Content
Back Button + Title	56px	Arrow back + 'Clock In' / '□□□□□□ □□□□'
GPS Status Card	80px	Shows current GPS status: 'Getting your location...' (loading), 'Location found: Near Greenfield Site' (success), 'Unable to get location' (error). Animated radar pulse while loading.
Time Display	100px	Large current time: '07:02 AM' in 36px bold. Date below: 'Monday, 15 March 2026' in 14px muted.
Confirm Button	80px	Full-width green button: 'CLOCK IN' / '□□□□□□ □□□□'. 56px height. Disabled until GPS is captured. Loading spinner when processing.
Info Text	40px	Small text: 'Your location will be recorded' / '□□□□ □□□□□□ □□□□□□ □□□□'

Interaction Flow

1. Worker taps 'Clock In' card on home screen → navigates to /clock-in
2. Page loads → immediately starts GPS capture (mock: 1-second simulated delay)
3. GPS status shows 'Getting your location...' with radar animation
4. GPS captured → status turns green: 'Location found: Near Greenfield Site' (reverse geocoded in Phase C, mock shows site name)
5. Worker taps the big green 'CLOCK IN' button
6. Button shows loading spinner for 0.5s (simulated write)
7. Success: Full-screen green checkmark overlay with 'Clock-in recorded!' / '□□□□□□ □□□□ □□ □□!' — auto-dismisses after 2 seconds
8. Redirects back to home screen. Status card now shows green 'Clock in since 7:02 AM'

Error States

Error	What Shows	Recovery
GPS permission denied	Red card: 'Please allow location access' with a 'Open Settings' button	Worker must go to phone settings and allow location. Page polls for permission change.
GPS timeout (30 seconds)	Amber card: 'Unable to get location. You can still clock in.' Button becomes enabled.	Allow clock-in without GPS. Record will be flagged for sub-manager review.
Network error (offline)	Orange banner: 'Offline mode'. Clock-in saves locally and syncs later.	In Phase A mock, simulate offline by storing in Zustand instead of mock API.

Component Tree

```

ClockInPage
├── PageHeader (back button + title)
├── GPSStatusCard (loading/success/error states)
├── TimeDisplay (current time, auto-updating every second)
├── ClockInButton (disabled until GPS ready, loading state on tap)
├── InfoText (location will be recorded)
└── SuccessOverlay (full-screen checkmark, auto-dismiss)

```

Mock Data Written

On successful clock-in, the page creates a new attendance record in the mock data store:

```

{ id: uuid(), tenantId: 'demo-tenant', workerId: currentUser.id,
  projectId: currentUser.projectId, date: '2026-03-15',
  clockIn: '2026-03-15T07:02:00', clockInLat: 19.0760,
  clockInLng: 72.8777, status: 'present', markedBy: currentUser.id,
  approvalStatus: 'auto-approved' }

```

Priority: P0 — Build immediately after Worker Home.

Estimated time: 3-4 hours.

2.3 Worker Photo Upload Page

URL: /photo

Purpose: Capture and upload a work photo with automatic GPS coordinates, timestamp, and worker identity. This replaces 'send photo to WhatsApp group'.

Frequency: 1-5 times per day. Workers snap progress photos, material delivery, work completion.

Screen Layout

Section	Content
Camera Viewfinder	Full-width camera preview (in Phase A, show a placeholder image with a 'Tap to simulate photo' overlay). In Phase C mobile, this uses react-native-vision-camera.
Capture Button	Large circle button at bottom center (like standard camera app). 64px diameter. White with subtle border.
GPS Overlay	Small badge in bottom-left of viewfinder showing 'GPS: 19.076, 72.877' in semi-transparent bar. Proves location is being captured.
Time Overlay	Small badge in bottom-right showing current time: '10:32 AM' in semi-transparent bar.
After Capture: Preview	Full-screen preview of captured photo with watermark overlay (date, time, GPS, worker name) burned in. Two buttons: 'Retake' and 'Upload'.
After Capture: Note Field	Optional text input below preview: 'Add a note (optional)' / '□□□ □□□□□□ (□□□□□□□□)'. Single line, max 200 characters.
Upload Button	Full-width blue button: 'Upload Photo' / '□□□□□ □□□□□ □□□□'. Shows upload progress bar if online, or 'Saved. Will upload when online.' if offline.

Photo Watermark Specification

Every photo gets a permanent watermark burned into the image. This is the core proof-of-work mechanism for construction documentation.

Element	Position	Format	Example
Date + Time	Bottom-left	DD MMM YYYY, HH:MM AM/PM	15 Mar 2026, 10:32 AM
Worker Name	Bottom-left (below date)	Full name	Ramesh Kumar
GPS Coordinates	Bottom-right	Lat, Lng (6 decimal)	19.076023, 72.877655
Project Name	Bottom-right (below GPS)	Site name	Greenfield Residency

Phase A Implementation (Web Mock)

Since we can't access the device camera in a web browser the same way as React Native, the Phase A web version uses a simulated camera flow:

1. Show a camera placeholder image (a stock construction site photo) with a 'Tap to Simulate Capture' button
2. On tap, select a random placeholder photo from a set of 5-6 pre-loaded construction images
3. Apply watermark overlay using HTML Canvas or CSS overlay (positioned text on the image)
4. Show the preview screen with watermarked photo + note field + upload button
5. On 'Upload', add the photo record to mock data store and show success overlay
6. In Phase C (mobile), this will use react-native-vision-camera + react-native-compressor + react-native-view-shot for real camera capture with native watermarking

Priority: P0 — Build in Week 1, right after Clock In.

Estimated time: 5-6 hours (including Canvas watermark implementation).

2.4 Worker View Task Page

URL: /task

Purpose: Read-only view of today's assigned task. Worker sees what they're supposed to do. Can mark it as complete.

Frequency: Checked once in the morning, once at completion.

Screen Layout

State	What Shows
Task assigned	Card with: Task title (bold, 18px), Description (14px muted), Assigned by (sub-manager name), Status badge ('Pending' amber / 'In Progress' blue / 'Completed' green). Below: full-width button 'Mark as Complete' / '□□□□ □□ □□□' (green). Button disabled if already completed.
No task assigned	Friendly illustration (simple construction worker SVG/icon) + text: 'No task assigned for today' / '□□ □□ □□□ □□□ □□□□'. Subtitle: 'Ask your site manager.' / '□□□□ □□□□ □□□□□□ □□ □□□□□'.
Task completed	Same card but with green 'Completed' badge and disabled button. Shows completion time: 'Completed at 3:45 PM'.

Priority: P1 — Build after Clock In and Photo Upload.

Estimated time: 2-3 hours.

2.5 Worker Attendance History Page

URL: /attendance

Purpose: Calendar view showing the worker's own attendance for the current month. Read-only — builds trust by letting workers verify their records.

Screen Layout

Section	Content
Month Header	Current month name + year ('March 2026' / '□□□□ 2026'). Left/right arrows to navigate months. Cannot go beyond current month.
Calendar Grid	Standard 7-column calendar (Sun-Sat). Each date cell: number + colored dot indicator. Green dot = Present, Red dot = Absent, Blue dot = Leave, Amber dot = Half-day, Gray = future date or no data.
Summary Card	Below calendar: 'Total Present: 22 days' (green), 'Absent: 3 days' (red), 'Leave: 2 days' (blue), 'Half-day: 1' (amber). Total hours if tracked.
Day Detail (on tap)	Tap any past date → expand to show: Clock-in time, Clock-out time, Total hours, GPS location of clock-in. No editing — read only.

Priority: P1 — Build after core action pages.

Estimated time: 4-5 hours (calendar component is complex).

2.6 Worker Profile Page

URL: /profile

Purpose: Display worker's identity information and provide settings access.

Estimated time: 2 hours. Simple read-only card with photo, name, phone, role, site, emergency contact.

2.7 Worker Leave Request Page

URL: /leave

Purpose: Submit a leave request. Simple form: date picker + reason dropdown + submit.

Estimated time: 2-3 hours.

Form Fields

Field	Type	Validation	Hindi Label
Start Date	Date picker (shadcn Calendar)	Required. Must be today or future.	□□□□ □□ □□□□□□
End Date	Date picker	Required. Must be >= start date.	□□□□□□ □□□□□□
Reason	Select dropdown	Required. Options: Sick / Personal / Family / Festival / Other	□□□□
Note	Textarea (optional)	Max 200 chars	□□□□□□□□ □□□□□□□□

Part 3: Driver Role — 8 Pages

The Driver interface is centered around vehicle operations. The critical difference from a Worker is the vehicle context — every action a Driver takes is tagged to a specific vehicle. The UI must adapt based on vehicle type: trucks show KM-focused forms, while heavy equipment (JCB, crane) shows hours-focused forms.

Driver Design Principles

- Vehicle context is always visible — the assigned vehicle's name and plate number appear in the header of every page.
- KM-type vs Hours-type distinction is the most important UI branching. The label changes, the unit changes, but the form structure stays the same.
- Odometer/meter photo is MANDATORY for start/end day logs. Cannot be skipped.
- Fuel entry always requires a receipt photo. This prevents fake fuel claims.
- Multi-driver support: the same vehicle might have 2-3 drivers per day. Each logs independently.

3.1 Driver Home Screen

URL: /home (within (driver) route group)

Purpose: Shows today's assigned vehicle, quick action buttons, and today's driving summary.

Screen Layout

Section	Content
Header	Driver name, profile photo, language toggle, notification bell
Vehicle Card	Prominent card showing: Vehicle name ('Tata 407'), Plate ('MH-04-AB-1234'), Type badge ('KM Tracking' or 'Hours Tracking'), Status dot (green = active). This is the FIRST thing the driver sees.
Today's Summary	Compact row: 'Today: 45 KM Rs.2,450 fuel 2 trips' (auto-calculated from today's logs). Shows '---' if no logs yet for today.
Action Cards (2x2 grid)	Start Day (green — disabled after logging), End Day (red — disabled until start logged), Log Fuel (amber), Log Expense (gray). Each card 120px+ height with icon + label.
Bottom Nav	4 tabs: Home, Fuel, Expense, Photos

Vehicle Type Branching

The entire Driver UI adapts based on whether the assigned vehicle is a 'km-type' or 'hours-type' vehicle. Here's what changes:

UI Element	KM-Type (Truck, Car)	Hours-Type (JCB, Crane, Excavator)
Vehicle card badge	'KM Tracking' (blue)	'Hours Tracking' (amber)
Start Day form label	'Enter starting KM reading'	'Enter starting hour meter reading'
End Day form label	'Enter ending KM reading'	'Enter ending hour meter reading'
Today's summary	'45 KM driven today'	'6.5 hours operated today'
Photo prompt	'Photograph the odometer'	'Photograph the hour meter'
Unit displayed	'KM'	'Hours'
Fuel efficiency	'KM per liter'	'Liters per hour'

Priority: P0 — Build in Week 2 of development.

Estimated time: 4-5 hours.

3.2 Driver Start Day Log

URL: /start-day

Purpose: Capture vehicle starting state: odometer/hour meter photo + numeric reading + GPS.

Form Fields

Field	Type	Validation	Notes
Meter Photo	Camera capture (mandatory)	Required. Must be captured in-app, not from gallery.	In Phase A: simulated camera with placeholder images
Reading	Number input	Required. Must be > 0. Must be > yesterday's end reading.	Label changes: 'KM Reading' or 'Hour Reading' based on vehicle type
GPS	Auto-captured	Captured automatically on page load. Shown as badge.	Same pattern as Worker clock-in GPS capture
Timestamp	Auto-captured	Current time, shown in large text above form	Not editable — prevents backdating

Validation Rules

- Reading must be numeric and > 0
- Reading must be $\geq$ vehicle's last known reading (prevents data entry errors like typing 45 instead of 45000)
- If reading is suspiciously high ($>$ last reading + 500 for KM, $>$ last reading + 24 for hours), show a warning: 'This seems high. Are you sure?' with Confirm/Edit buttons
- Photo is mandatory — submit button stays disabled until photo is captured
- GPS is captured automatically but is NOT required for submission (some vehicles may be in covered areas)

Priority: P0 — Core driver functionality.

Estimated time: 4-5 hours (including photo simulation).

3.3 Driver End Day Log

URL: /end-day

Purpose: Same form as Start Day but captures ending state. Auto-calculates today's total.

Key difference: After submitting, shows a summary card: 'Today you drove 87 KM' or 'Today the machine ran 6.5 hours'. This gives the driver immediate feedback.

Estimated time: 3 hours (reuses Start Day form component with different labels).

3.4 Driver Fuel Entry

URL: /fuel

Purpose: Log a fuel fill-up with amount, liters, and mandatory receipt photo.

Form Fields

Field	Type	Validation
Amount (Rs.)	Number input with Rs. prefix	Required. Must be > 0.
Liters	Number input with 'L' suffix	Required. Must be > 0.
Receipt Photo	Camera capture	MANDATORY. Cannot submit without receipt.
GPS	Auto-captured	Auto. Proves driver was at a fuel station.
Note	Text input (optional)	Max 100 chars. 'Which pump, etc.'

Auto-calculated: 'Cost per liter: Rs.XX.XX' shown below the Amount and Liters fields in real-time as the user types.

Priority: P0.

Estimated time: 3-4 hours.

3.5 Driver Expense Entry

URL: /expense

Purpose: Log non-fuel expenses (toll, parking, repair, food).

Form: Amount + Category dropdown (Toll/Parking/Repair/Food/Other) + Optional receipt photo + Optional note.

Estimated time: 2-3 hours (simpler than fuel entry).

3.6 Driver Vehicle Issue Report

URL: /issue

Purpose: Report mechanical problems. Critical severity triggers push notification to Manager.

Form: Issue type dropdown + Description + Photo(s) + Severity selector (Can continue / Needs attention / Breakdown).

Special: Selecting 'Breakdown' shows a red warning: 'This will immediately alert your manager' before submission.

Estimated time: 3-4 hours.

3.7 Driver Daily Summary

URL: /summary

Purpose: Read-only auto-generated summary of today's driving activity.

Content: Total KM/Hours, Start reading → End reading, Fuel consumed (liters + Rs.), Expenses logged, Number of trips. All auto-calculated from the day's log entries.

Estimated time: 2-3 hours.

3.8 Driver Photo Upload

URL: /photos

Purpose: Upload trip-related photos (loading, unloading, delivery proof). Same photo pipeline as Worker but tagged to vehicle + trip context.

Estimated time: 2 hours (reuses Worker photo component).

Part 4: Worker-Head (Mistri) Role — 8 Pages

The Worker-Head is the most unique role in techBuilder. This person is a senior, trusted construction worker (Mistri) who acts as a bridge between the ground crew and management. They do everything a worker does PLUS manage their crew's attendance, upload data on behalf of workers without phones, and report progress.

Worker-Head Design Principles

- Inherits ALL Worker features (clock in/out, photo upload, tasks) — the bottom nav includes worker actions.
- The 'Upload on Behalf' feature is the killer differentiator — it requires a person-picker before every action.
- Crew management is the primary view — who showed up today, who didn't, is front and center.
- Material usage logging is important for tracking consumption vs purchases (detecting theft/waste).
- Daily note replaces the WhatsApp evening message. Keep it simple: just a text box.

4.1 Worker-Head Home Screen

URL: /home (within (worker-head) route group)

Purpose: Shows crew status, quick actions for crew management + all worker actions.

Screen Layout

Section	Content
Header	Mistri name + 'Crew Lead' badge + language toggle + notification bell
Crew Summary Card	Prominent card: 'Your Crew: 12 workers. Today: 10 present, 1 absent, 1 leave'. Tap to see full crew list. Green/red mini dots for visual scan.
Action Cards (2x3 grid)	6 cards: Mark Attendance (green), Upload for Others (blue), Progress Photo (purple), Material Log (amber), Report Issue (red), Daily Note (gray). Each 80-100px height.
Self Actions Row	Smaller row: Your Clock In/Out, Your Photo. Same as worker actions but deprioritized visually.
Bottom Nav	4 tabs: Home, Crew, Upload, Report

Priority: P0 — Build in Week 3.

Estimated time: 5-6 hours.

4.2 Worker-Head Crew Attendance Page

URL: /crew-attendance

Purpose: Mark attendance for all crew members. This is the digital replacement for the paper attendance register.

Screen Layout

Section	Content
Date Header	Today's date, large and prominent. Cannot change date (prevents backdating).
Bulk Actions	Row of buttons: 'All Present' (marks everyone), 'Reset' (clears all). Saves time for Mistri.
Worker List	Scrollable list of crew members. Each row: Photo (40px circle), Name (bold), Phone (muted), Three-way toggle: Present (green) / Absent (red) / Half-Day (amber). Toggle is a segmented control — large enough to tap with gloves.
Submit Button	Full-width green button: 'Submit Attendance for 12 workers'. Shows count of how many are marked. Disabled until at least one is marked.
Confirmation	After submit: success overlay with 'Attendance submitted for 12 workers. 10 present, 1 absent, 1 half-day.' with checkmark animation.

Key UX: The 'All Present' button saves enormous time — most days, all workers show up. Mistri taps 'All Present', then changes the 1-2 who are absent/half-day. 3 taps instead of 12.

Priority: P0.

Estimated time: 5-6 hours (list + toggle + bulk action + submit).

4.3 Worker-Head Upload on Behalf

URL: /upload-behalf

Purpose: Upload photos, mark tasks, or log data on behalf of a worker who does not have a phone.

Interaction Flow

1. Page shows a crew member picker: list of all crew members with photos and names.
2. Mistri selects the worker they are acting for. Selected worker's name appears in a blue banner at top: 'Uploading for: Ramesh Kumar'.
3. Action picker appears: 'What do you want to do for Ramesh?' — Upload Photo, Mark Task Complete, Log Note.
4. Selected action opens the SAME form as the worker would see, but the data is tagged with: uploadedBy = Mistri's ID, onBehalfOf = Ramesh's ID.
5. After submission, success message: 'Photo uploaded for Ramesh Kumar' with dual attribution shown.
6. Banner clears, ready for next person or next action.

Priority: P0 — This is techBuilder's killer differentiator.

Estimated time: 6-8 hours (person picker + conditional routing + dual attribution).

4.4-4.8 Remaining Worker-Head Pages

Page	URL	Purpose	Est. Time
Progress Report	/progress	Upload progress photo with description text + progress % slider + milestone tag	4 hours
Material Log	/material	Select material type (dropdown), enter quantity + unit, optional photo. Running daily total shown.	3 hours
Issue Report	/issues	Same as driver issue report but tagged to project (not vehicle). Photos + severity + category.	3 hours (reuse driver component)
Daily Note	/daily-note	Simple text area for evening summary. Timestamp auto-added. Submit saves to daily reports.	2 hours
Expense Entry	/expense	Same expense form as driver but project-tagged. Amount + category + optional receipt photo.	2 hours (reuse driver component)

Part 5: Sub-Manager Role — 11 Pages

The Sub-Manager is the site-level decision maker. They manage one construction site with 20-100 workers, multiple worker-heads, and several vehicles. Their interface is the most complex of the field roles — it combines data consumption (viewing attendance, expenses, progress across their site) with data creation (daily reports, task assignment, expense approval). This role is typically viewed on a mid-range smartphone (Rs.10,000-20,000) or occasionally a tablet.

Sub-Manager Design Principles

- Site-centric: Everything is filtered to their one assigned project. They never see other sites.
- Approval workflows: They are the first approval gate for attendance and expenses.
- Daily report is their primary evening responsibility — must be fast to complete.
- Task assignment is lightweight — no project management complexity, just daily task lists.
- Information density is higher than worker/driver — sub-managers can handle more data on screen.

5.1 Sub-Manager Home / Site Dashboard

URL: /home (within (sub-manager) route group)

Purpose: Overview of their site: today's attendance, expenses, recent activity, pending approvals.

Screen Layout

Section	Content
Header	Sub-Manager name + Site name badge ('Greenfield Residency') + notifications
KPI Row	4 mini KPI cards in a row: Workers Today (22/25), Expenses Today (Rs.12,500), Issues Open (3), Tasks Pending (5). Each tappable to drill down.
Pending Approvals Banner	If any pending attendance approvals or expense approvals: amber banner: '4 items need your approval' → taps to approval queue. Badge count.
Recent Activity Feed	Last 10 activities at this site: clock-ins, photos uploaded, expenses submitted, issues reported. Compact list with timestamp + description + type icon.
Quick Actions	Floating action button (FAB) at bottom-right with expandable menu: Upload Photo, Create Task, Log Expense, Write Report.
Bottom Nav	5 tabs: Home, Attendance, Tasks, Expenses, More (hamburger for remaining pages)

Priority: P0 — Build in Week 4.

Estimated time: 6-8 hours (most data-dense page for field roles).

5.2 Sub-Manager Attendance Page

URL: /attendance

Purpose: View and manage attendance for all workers on their site. Two modes: approve pending clock-ins, or take attendance directly.

Approval Mode

When workers self-clock-in, their entries appear here for approval. Sub-manager sees: Worker name, Clock-in time, GPS location (distance from site), and Approve/Reject buttons.

Direct Mode

Same as Worker-Head crew attendance but for ALL workers on the site (not just one crew). Bulk 'All Present' + individual toggles.

Priority: P0.

Estimated time: 5-6 hours (two modes + approval workflow).

5.3-5.11 Remaining Sub-Manager Pages

Page	URL	Purpose	Key Components	Est. Time
Daily Photo Upload	/photos	Morning + evening site photos with GPS watermark	Photo capture + time-of-day tag	3 hours
Evening Report	/report	Daily summary with auto-filled data + free text	Auto-fill from day's data + text area + photo attachment	5 hours
Task Management	/tasks	Create, assign, and track daily tasks for workers	Task form + worker picker + status tracker + list view	6 hours
Expense Management	/expenses	Log expenses + approve worker-head submissions	Expense form + approval list with receipt photos	5 hours
Progress Tracker	/progress	Update completion % per milestone	Milestone list + progress slider + photo evidence	4 hours
Worker List	/workers	View all workers on site with contact + attendance summary	Searchable list + worker cards + quick actions	3 hours
Vehicle View	/vehicles	See vehicles assigned to site (read-only)	Vehicle cards with today's stats	2 hours
Issue Tracker	/issues	View, respond to, and escalate site issues	Issue list + status filter + escalate button	4 hours
Material Request	/material-request	Request materials from main office	Material picker + quantity + urgency + request status list	3 hours

Part 6: Manager (Company Owner) Role — 14 Pages

The Manager sees everything. This is the most data-intensive, feature-rich part of the application. The Manager interface must aggregate data from ALL sites, ALL workers, ALL vehicles, and ALL expenses into actionable dashboards. This role is typically used on desktop (50%) and phone (50%).

Manager Design Principles

- Desktop-first design (unlike all other roles which are mobile-first). Sidebar navigation on desktop, bottom nav on mobile.
- Data density is maximum — tables, charts, multiple KPIs, filterable lists.
- Cross-site aggregation: every view shows data from ALL sites with ability to filter by site.
- Approval is a primary action — expenses, attendance, issues all flow to the Manager for final approval.
- Reports and exports are critical — this user downloads PDFs and Excels regularly.
- Activity feed is the 'WhatsApp killer' — replacing 5 WhatsApp groups with one structured, filterable feed.

6.1 Manager Dashboard (Home)

URL: /home (within (manager) route group)

Purpose: Company-wide overview with KPIs, charts, and activity feed. THE most important page in the app.

Desktop Layout (3-column)

Column	Width	Content
Left Sidebar	240px	Navigation: Dashboard, Sites, Attendance, Fleet, Feed, Expenses, Approve, Workers, Vehicles, Reports, Assign, Notifications. Active item highlighted. Collapse to icons on narrow screens.
Main Content	Flex	KPI Cards row (4 cards) → Expense chart (last 7 days, bar chart) → Attendance trend (last 7 days, line chart) → Recent issues (compact list)
Right Panel	320px	Activity feed (scrollable). Pending approvals count. Quick stats.

Mobile Layout (Single Column)

On mobile, content stacks vertically: KPI cards (2x2 grid) → Charts (full-width, swipeable) → Activity feed → Bottom nav (Home, Sites, Approve, Feed, More).

KPI Cards

KPI	Value Example	Trend	Tap Action
Workers Today	152 / 180 (84%)	Up 3% from yesterday (green arrow)	Navigate to Attendance page
Active Vehicles	12 / 15	Same as yesterday (gray dash)	Navigate to Fleet page
Today's Expense	Rs. 45,200	Down 12% from yesterday (green arrow)	Navigate to Expenses page
Active Sites	5	No trend	Navigate to Sites page

Charts

Two charts on the dashboard, both using Recharts:

Chart	Type	Data	Interaction
Expense Trend	Bar chart	Last 7 days total expenses. Color by category (material=blue, fuel=amber, labor=green, other=gray)	Tap bar to see day detail. Toggle between 7-day, 30-day views.
Attendance Trend	Line chart	Last 7 days attendance rate (%). Green line for present rate.	Hover/tap to see exact count. Per-site breakdown on tap.

Priority: P0 — The hero page of the entire product. Build in Week 5.

Estimated time: 10-12 hours (most complex page with charts, KPIs, responsive layout, activity feed).

6.2-6.14 Remaining Manager Pages

#	Page	URL	Purpose	Key Components	Est. Time
6.2	All Sites	/sites	Grid/list of all active projects with status, worker count, progress %	Site cards + status badges + search/filter	4 hours
6.3	Site Detail	/sites/[id]	Deep-dive into one site: workers, vehicles, expenses, photos, progress	Tabbed interface with 5 tabs per site	6 hours
6.4	Attendance Overview	/attendance	Cross-site attendance: total present/absent, per-site breakdown, trends	Summary cards + per-site table + date picker	5 hours
6.5	Fleet Overview	/fleet	All vehicles: today's KM/hours, fuel, driver, status	Vehicle cards with today's stats + status filter	4 hours
6.6	Activity Feed	/feed	Chronological feed from all sites. Filter by site, type, role.	Infinite scroll + filter tabs + photo previews	6 hours
6.7	Expense Dashboard	/expenses	Total spending: today, week, month. Category breakdown chart.	KPI cards + pie chart + bar chart + date range	5 hours
6.8	Approve Queue	/approve	Pending expenses and attendance for approval. Batch approve.	Approval list with receipt photos + batch actions	5 hours
6.9	Worker Directory	/workers	All workers across all sites. Search, filter, reassign.	Data table (TanStack) + search + role/site filters	5 hours
6.10	Vehicle Directory	/vehicles	All vehicles with documents, maintenance, assignment.	Data table + document viewer + reassign action	4 hours
6.11	Reports & Downloads	/reports	Generate and download PDF/Excel reports.	Report type selector + date range + site filter + download buttons	6 hours

#	Page	URL	Purpose	Key Components	Est. Time
6.12	Assign Workers	/assign/workers	Move workers between sites.	Source site picker + worker multi-select + target site	4 hours
6.13	Assign Vehicles	/assign/vehicles	Assign vehicles to sites and drivers.	Vehicle picker + site/driver assignment form	3 hours
6.14	Notifications	/notifications	All alerts and notifications.	Notification list + type filter + read/unread toggle	3 hours

Part 7: Super Admin & Shared Pages

7.1 Super Admin — 5 Pages (Desktop Only)

The Super Admin interface is YOUR dashboard as the platform owner. It shows all tenant companies, revenue, and system health. Desktop-only — no mobile optimization needed.

Page	URL	Purpose	Est. Time
Platform Dashboard	/home	Total tenants, total users, MRR, growth charts, system health	5 hours
Tenant Management	/tenants	List, create, suspend, delete companies. Usage metrics per tenant.	5 hours
Billing & Subscriptions	/billing	Razorpay integration UI. Plans, payment history, failed payments.	4 hours (UI only, Razorpay in Phase C)
Audit Log	/audit	Immutable log: who did what, when. Filterable.	3 hours
Backups	/backups	Backup status monitoring. Last backup time, size, restore button.	2 hours

7.2 Shared Pages — Used by All Roles

Page	URL	Purpose	Est. Time
Login	/login	Phone OTP + Email/password. Language selector. Company code entry.	6 hours (complex: dual auth method)
Signup / Onboarding	/signup	Company creation wizard: company name, logo, first project, invite team.	6 hours (multi-step wizard)
Dev Role Selector	/dev/select-role	Development-only page to switch between roles. 6 role buttons.	1 hour
GPS Consent	/consent	Legal consent screen before GPS tracking starts. Hindi/English.	2 hours
Photo Gallery	/gallery	Browse project photos by date. Grid view with lazy loading.	4 hours
Notification Center	/notifications	All notifications with read/unread and type filter.	3 hours
Settings	/settings	Language toggle, notification preferences.	2 hours

Part 8: Implementation Schedule — Week-by-Week

This is the build order for Phase A Frontend. Total: approximately 8-10 weeks for a single senior developer, or 5-6 weeks with two developers.

Week	Focus	Pages Built	Estimated Hours
Week 1	Project Setup + Worker Core	Setup (monorepo, Tailwind, shadcn, i18n, mock data structure) + Worker Home + Clock In + Clock Out + Photo Upload	35-40 hours
Week 2	Worker Complete + Driver Start	Worker Task + Attendance + Profile + Leave + Driver Home + Start Day + End Day	35-40 hours
Week 3	Driver Complete + Worker-Head Start	Driver Fuel + Expense + Issue + Summary + Photos + Worker-Head Home + Crew Attendance	35-40 hours
Week 4	Worker-Head Complete + Sub-Manager Start	Worker-Head Upload-Behalf + Progress + Material + Issues + Daily Note + Sub-Manager Home + Attendance	35-40 hours
Week 5	Sub-Manager Complete	Sub-Manager Photos + Report + Tasks + Expenses + Progress + Workers + Vehicles + Issues + Material Request	35-40 hours
Week 6	Manager Dashboard + Core	Manager Home (KPIs, charts) + All Sites + Site Detail + Attendance Overview + Fleet Overview	40-45 hours
Week 7	Manager Complete	Manager Feed + Expenses + Approve + Workers + Vehicles + Reports + Assign Workers + Assign Vehicles + Notifications	35-40 hours
Week 8	Admin + Shared + Polish	Super Admin (5 pages) + Login + Signup + Consent + Gallery + Notification Center + Settings + Dev Role Selector	35-40 hours
Week 9	Hindi Translation + Responsive	Complete Hindi translations for all pages + responsive testing on all breakpoints + fix layout issues	30-35 hours
Week 10	UI Polish + Testing	Loading states, empty states, error states for every page. Skeleton screens. Accessibility audit. Cross-browser testing.	30-35 hours

Total Estimates Summary

Role	Pages	Estimated Hours	Percentage of Total
Worker	6	20-25 hours	7%
Driver	8	25-30 hours	9%
Worker-Head	8	25-30 hours	9%
Sub-Manager	11	40-45 hours	14%
Manager	14	60-70 hours	22%
Super Admin	5	19-22 hours	7%
Shared Pages	7	24-28 hours	9%
Setup + Design System	—	25-30 hours	9%
Hindi + Responsive + Polish	—	40-45 hours	14%
TOTAL	59 pages	280-325 hours	100%

Part 9: Shared Component Library

These are the reusable components built during Phase A that will be used across multiple pages and roles. Building them as standalone, well-documented components saves significant time.

Component	Used By	Props	Description
ActionCard	Worker, Driver, Worker-Head home screens	icon, label, color, subtitle, disabled, badge, onClick	Large tappable card (120px height) with icon + text. Supports disabled state, badge count (notifications).
KPICard	Sub-Manager home, Manager dashboard	value, label, trend, trendDirection, color, onClick	Compact metric card. Shows number, label, and optional trend arrow. Tappable.
AttendanceStatusCard	Worker home	status, clockInTime, totalHours	Displays current attendance state with color-coded background.
WorkerCard	Crew lists, worker directory	photo, name, role, status, attendanceRate, onClick	Horizontal card with avatar, name, role badge, and status indicator.
VehicleCard	Driver home, fleet views	name, plate, type, todayKm, todayHours, status, driverName	Vehicle display card with type badge and today's stats.
ExpenseCard	Expense lists, approval queue	amount, category, submitter, date,	Expense display with category icon, amount, and status badge.

Component	Used By	Props	Description
		receiptPhoto, approvalStatus	
PhotoCapture	Worker photo, Driver meter photo, progress photos	onCapture, watermarkData, required	Camera simulation (Phase A) / real camera (Phase C). Handles watermark overlay.
GPSStatusBadge	Clock-in, photo upload, fuel entry	status, lat, lng, locationName	Shows GPS capture state: loading (pulse), success (green), error (red).
ApprovalButtons	Expense approval, attendance approval	onApprove, onReject, itemId, loading	Approve (green) + Reject (red) button pair with loading state.
ActivityFeedItem	Manager feed, Sub-Manager feed	type, description, timestamp, photo, userName, siteName	Single activity entry with type icon, description, and optional photo thumbnail.
DateRangePicker	Reports, analytics, expense views	startDate, endDate, onChange, presets	shadcn Calendar-based range picker with preset options (Today, This Week, This Month).
EmptyState	All list views	icon, title, subtitle, actionLabel, onAction	Friendly empty state with illustration, message, and optional CTA button.
OfflineBanner	All pages	isOffline, pendingCount	Orange banner at top: 'You are offline. X changes pending.' Auto-shows when offline.
SuccessOverlay	Clock-in, submissions, approvals	message, onDismiss, autoDismissMs	Full-screen green checkmark overlay that auto-dismisses.
BottomNav	All mobile role layouts	items, activeItem, role	Role-specific bottom navigation bar. 3-5 tabs depending on role.
Sidebar	Manager, Admin desktop layouts	items, activeItem, collapsed	Desktop sidebar navigation. Collapsible to icon-only mode.
PageHeader	All pages	title, backButton, actions	Consistent page header with title, optional back button, and action buttons.
LoadingSkeleton	All pages	type: card list chart form	Skeleton loading placeholders matching the page layout.

Part 10: Testing & Quality Assurance

10.1 Device Testing Matrix

techBuilder's primary users are Indian construction workers. Testing must cover the devices they actually use.

Device Category	Example Models	Screen Size	Priority	Expected Users
Budget Android	Redmi 9A, Realme C35, Samsung A03	5.0-6.5 inch, HD+	P0 — HIGHEST	70% of workers
Mid-range Android	Redmi Note 12, Samsung M34, Poco X5	6.4-6.7 inch, FHD+	P0	25% of workers + sub-managers
Tablet	Samsung Tab A8, Lenovo Tab M10	10.1 inch	P1	Some site managers
Desktop Chrome	Any modern Chrome/Edge	1280-1920px	P0 for Manager/Admin	Managers, Admin
iPhone	iPhone 12/13/14	6.1 inch	P2	Rare — < 5% of construction workers in India

10.2 Quality Checklist Per Page

Before marking any page as 'complete', verify ALL items on this checklist:

- Renders correctly on 360px width (minimum phone width)
- Renders correctly on 1280px+ width (desktop)
- Hindi translation complete — no hardcoded English strings visible when switched to Hindi
- Loading state implemented (skeleton or spinner)
- Empty state implemented (friendly message when no data)
- Error state implemented (red toast or error card when mock data fails)
- All touch targets are minimum 44x44px
- Color contrast ratio passes 4.5:1 (use Chrome DevTools Accessibility panel)
- Keyboard navigation works (Tab through all interactive elements)
- Screen reader announces meaningful content (test with Chrome Accessibility tree)
- No horizontal scroll on any screen width
- Bottom navigation does not overlap content (pb-20 padding)
- Page loads in under 2 seconds on throttled 3G (Chrome DevTools Network throttling)
- No console errors or warnings in browser console

- Mock data produces realistic-looking UI (not obviously fake)

Part 11: Conclusion — What You Have Now

This document defines the complete Phase A frontend for techBuilder:

- 59 unique pages across 6 user roles + shared pages
- 14 mock data entities with exact field schemas
- 18 shared reusable components
- Complete design system (colors, typography, spacing, patterns)
- Role-based routing architecture with middleware
- Hindi + English translation structure for every page
- Week-by-week implementation schedule (8-10 weeks solo, 5-6 weeks with 2 devs)
- 280-325 estimated development hours
- Device testing matrix targeting Indian construction workers
- Quality checklist ensuring production readiness

The service layer pattern (`lib/api.ts`) ensures that when Phase B (backend integration) begins, page components will NOT change. Only the data-fetching functions switch from mock JSON imports to real HTTP requests. This is the key architectural decision that makes Phase A work as both a design validation AND a production foundation.

— End of Phase A Frontend Implementation Guide —